MFJA 3rd Floor Gazebo Harmonic Final Multi-Robot and Meta-Repository Integration Report

Technical Project Report

March 25, 2026

Contents

1 Introduction **3**

2 Final Result and Scope **4**
2.1 Delivered final system 4
2.2 Why this structure was necessary 5
2.3 Meta-repository package split 5
2.4 Full-floor and room-315-only modes 6
2.5 Final robot registries 6

3 Step-by-Step Development Path **8**
3.1 Step 1: Converting the KUKA robot into a usable Gazebo model 8
3.2 Step 2: Generalizing from one robot to a reusable launch architecture . . 9
3.3 Step 3: Building per-robot ROS–Gazebo bridges 9
3.4 Step 4: Spawning robots automatically from the registry 11
3.5 Step 5: Adding the Stäubli TX2-60L 12
3.6 Step 6: Adding Yaskawa HC10 and HC10DT as distinct robots 12
3.7 Step 7: Adding TIAGo first as a manipulator 12
3.8 Step 8: Upgrading TIAGo to a real mobile manipulator 13
3.9 Step 9: Enriching the world with industrial station assets 14
3.10 Step 10: Adding dedicated room assets and a room-315-only mode 14
3.11 Step 11: Refactoring the repository into five ROS 2 packages 15
3.12 Step 12: Freezing the final scope 15

4 Important Final Code and Why It Matters **17**
4.1 Code block 1: The configuration file 17
4.2 Code block 2: Bridge generation 17
4.3 Code block 3: Spawn loop and state publishers 18
4.4 Code block 4: KUKA articulation 18
4.5 Code block 5: TIAGo mobility support 18
4.6 Code block 6: Static world composition 19
4.7 Code block 7: Launch-time robot selection and synchronized demos . . . 19

5 How Independent Control Works **20**
5.1 Namespacing strategy 20
5.2 Why one state publisher per robot was necessary 20
5.3 Why TIAGo needed extra logic 20

6 Validation and User Operation Guide **22**
6.1 What was validated in the final system 22
6.2 How to run the program 22
6.3 How to verify that the system is ready 24
6.4 How to inspect the synchronized multi-robot tool 24
6.5 How to choose between one-by-one control and synchronized control . . . 24
6.6 How to control the industrial robots 25
6.7 How to control TIAGo 26
6.8 How to move one robot at a time 27
6.9 How to command several robots together from one node 28
6.10 How to change robot positions and active robots 30
6.11 Typical issues and final operational notes 31

7 Conclusion **32**

Chapter 1

Introduction

This report documents the final engineering work carried out on the meta-repository `mfja_3rd_floor_gz`. The objective was not only to turn the existing MFJA third-floor Gazebo Harmonic map into a reusable multi-robot simulation workspace, but also to reorganize the codebase into several ROS 2 packages inside the same Git repository. The final workspace supports five independently controllable robots:

- `kuka1`: KUKA KR6 R900 sixx;
- `staubli1`: Stäubli TX2-60L;
- `yaskawa_hc10_1`: Yaskawa HC10;
- `yaskawa_hc10dt_1`: Yaskawa HC10DT;
- `tiago1`: TIAGo mobile manipulator.

The work did not stop at inserting robot geometry into the scene. Each robot had to be spawnable from a shared launch pipeline, placed reproducibly inside the world, connected to ROS 2 with its own namespace, and controllable without topic collisions. In addition, the map was enriched with static industrial station assets such as robot tables, covers, rails, shuttle parts, and dedicated room assets.

This report explains the final solution only. It focuses on the changes that remained in the accepted configuration, the important code fragments that make the system work, and the development path that led from a single-robot KUKA setup to a five-robot mixed environment containing both industrial manipulators and one mobile manipulator. It also explains the later meta-repository refactor that separated shared assets, shared control logic, full-floor bringup, and room-315-only bringup into different ROS 2 packages while preserving the overall functionality.

Chapter 2

Final Result and Scope

2.1 Delivered final system

The final project delivers a configuration-driven simulation architecture with the following properties:

- five ROS 2 packages inside one Git repository instead of one monolithic package;
- two YAML registries for robot identity, model selection, pose, and activation: one for the full floor and one for room 315 only;
- one shared multi-robot launch file that reads the registry and spawns enabled robots;
- one full-floor bringup package and one room-315-only bringup package;
- one umbrella compatibility package that keeps simple launch commands available under the historical project name;
- one launch-time robot-selection argument that can override the YAML activation set from the command line using full names, short aliases, or numeric shortcuts;
- one ROS–Gazebo bridge configuration generated per robot at runtime;
- one `robot_state_publisher` per robot;
- articulated SDF models for KUKA, Stäubli, Yaskawa HC10, Yaskawa HC10DT, and TIAGo;
- TIAGo support as both a manipulator and a mobile base;
- one synchronized multi-robot command-line tool for coordinated motion scenarios;
- a full third-floor world plus a dedicated room-315-only world;
- a world file extended with station tables and industrial support parts.

2.2 Why this structure was necessary

At the beginning, the requirement was only to add a KUKA robot. Very quickly, the real requirement became broader:

- more robots would be added later;
- every robot had to remain independently controllable;
- placements had to stay reproducible;
- the system had to scale without rewriting the launch logic each time.

Because of that, a hard-coded single-robot launch solution would have been the wrong architecture. The final implementation therefore separates responsibilities clearly:

- `mfja_3rd_floor_description` stores shared Gazebo models, meshes, URDF files, and world files;
- `mfja_robot_control_config/config/robots.yaml` and `mfja_robot_control_config/config` decide *which* robots exist and where;
- `mfja_robot_control_config/launch/multi_robot_sim.launch.py` decides *how* robots are spawned and bridged, and can override the selected robot set from the command line;
- `mfja_3rd_floor_bringup` exposes the full-floor entry points;
- `mfja_room_315_bringup` exposes the isolated room-315-only entry point;
- `mfja_robot_control_config/scripts/multi_robot_sync_demo.py` coordinates simultaneous motion commands.

2.3 Meta-repository package split

The final repository is no longer one large ROS 2 package. Instead, it is organized as five packages in the same Git repository:

- `mfja_3rd_floor_gz`: umbrella and compatibility package that forwards common launch entry points while preserving a subpackage with the same name as the parent Git repository;
- `mfja_3rd_floor_description`: shared meshes, Gazebo models, URDF files, and world files;
- `mfja_robot_control_config`: shared launch logic, robot YAML files, shared configuration, and synchronized command tools;
- `mfja_3rd_floor_bringup`: launch entry points for the complete third-floor simulation;
- `mfja_room_315_bringup`: launch entry point for the room-315-only simulation.

This split matters technically because it reduces future merge conflicts. A developer adjusting shared environment assets can work mainly in `mfja_3rd_floor_description`. A developer changing robot-selection logic or synchronized control behavior can work in `mfja_robot_control_config`. Another developer working only on room 315 can concentrate on `mfja_room_315_bringup` without modifying the full-floor bringup package.

2.4 Full-floor and room-315-only modes

The final repository now supports two independent run modes:

- **Full third floor**: launched from `mfja_3rd_floor_bringup/full_floor.launch.py`;
- **Room 315 only**: launched from `mfja_room_315_bringup/room_315_only.launch.py`.

The full-floor mode uses the complete MFJA third-floor world and the main robot registry in `mfja_robot_control_config/config/robots.yaml`. The room-315-only mode uses the dedicated room 315 world and the separate registry in `mfja_robot_control_config/config/r`
This second mode keeps the scene cleaner for robot and control experiments while still reusing the same shared robot models and the same generic spawn logic.

2.5 Final robot registries

The final robot lists are stored in two YAML files:

- `mfja_robot_control_config/config/robots.yaml` for the complete third-floor mode;
- `mfja_robot_control_config/config/robots_room_315_only.yaml` for the isolated room-315-only mode.

The first file became the operational entry point for the complete laboratory layout. The second file preserves the same structure but allows room 315 experiments to evolve independently without changing the full-floor deployment.

```yaml
robots:
  - name: kuka1
    model: kuka_kr6r900sixx
    x_pose: -11.2
    y_pose: -6.0
    z_pose: 1.0
    yaw: 3.14
    enabled: true
  - name: staubli1
    model: staubli_tx2_60l
    x_pose: -16.0
    y_pose: -6.0
    z_pose: 1.0
    yaw: 1.57
    enabled: true
  - name: yaskawa_hc10_1
    model: yaskawa_hc10
    x_pose: -11
    y_pose: -3.0
    z_pose: 0.62
    yaw: 1.57
    enabled: true
  - name: yaskawa_hc10dt_1
    model: yaskawa_hc10dt
    x_pose: -16
    y_pose: -3.0
    z_pose: 0.62
    yaw: 1.57
    enabled: true
  - name: tiago1
    model: tiago
    x_pose: -6.4
    y_pose: -3.0
    z_pose: 0.0
    yaw: 1.57
    enabled: true
```

Listing 2.1: Main full-floor robot registry in `mfja_robot_control_config/config/robots.yaml`.

These YAML files are among the most important outcomes of the project because they converted robot deployment into data rather than hard-coded Python statements. A user can now change the active layout of the full floor or of room 315 by editing YAML fields instead of modifying the launch implementation itself.

Chapter 3

Step-by-Step Development Path

3.1 Step 1: Converting the KUKA robot into a usable Gazebo model

The first working milestone was the KUKA KR6 R900 sixx. The task was not solved by importing a mesh alone. A complete articulated model was needed, including:

- links with visual and collision geometry;
- inertial properties for simulation stability;
- six revolute joints with limits and damping;
- a joint-state publisher plugin;
- a joint-trajectory controller plugin.

One of the critical corrections was the use of explicit poses relative to joints. This is the change that eliminated the broken-arm behavior and missing or separated links that appeared in earlier attempts. The following excerpt shows the final link definitions for the first moving segments:

```
    <link name="link_1">
      <pose relative_to="joint_a1">0 0 0 0 0 0</pose>
      <inertial>
        <pose>0.0116 -0.0010 -0.0483 0 0 0</pose>
        <mass>8.0</mass>
        <inertia>
          <ixx>0.07</ixx>
          <iyy>0.07</iyy>
          <izz>0.05</izz>
          <ixy>0.0</ixy>
          <ixz>0.0</ixz>
          <iyz>0.0</iyz>
        </inertia>
      </inertial>
      <visual name="link_1_visual">
        <geometry>
          <mesh>
            <uri>model://kuka_kr6r900sixx/meshes/converted_visual/link_1.stl</uri>
          </mesh>
```

Listing 3.1: KUKA links attached with explicit poses relative to their joints.

The joint chain then defines the actual kinematic behavior. The final implementation uses explicit parent-child relations, fixed placement offsets, motion limits, damping, and friction:

```
    <joint name="joint_a1" type="revolute">
      <parent>base_link</parent>
      <child>link_1</child>
      <pose relative_to="base_link">0 0 0.400 0 0 0</pose>
      <axis>
        <xyz>0 0 -1</xyz>
        <limit>
          <lower>-2.96706</lower>
          <upper>2.96706</upper>
          <effort>800</effort>
          <velocity>6.28319</velocity>
        </limit>
        <dynamics>
          <damping>0.6</damping>
          <friction>0.05</friction>
        </dynamics>
      </axis>
    </joint>
```

Listing 3.2: KUKA revolute joint chain used in the final model.

The KUKA phase established the core pattern later reused by the other industrial arms: SDF for simulation, URDF for TF publication, plugin-based control inside Gazebo, and ROS topic access through bridging.

3.2 Step 2: Generalizing from one robot to a reusable launch architecture

After KUKA worked by itself, the next engineering problem was scalability. Adding every new robot by writing another dedicated spawn node would have created a fragile and repetitive launch file. The solution was to convert the launch process into a generic pipeline driven by the YAML registry.

The first part of that pipeline validates the enabled robots from YAML and resolves their SDF and URDF assets from a consistent package structure. This was a decisive design improvement. It enforced one package convention for all robots:

- `mfja_3rd_floor_description/models/<model_name>/model.sdf`
- `mfja_3rd_floor_description/urdf/<model_name>.urdf`

As a result, later robot additions did not require changing the launch structure. They only required adding correct model assets and one new YAML entry.

3.3 Step 3: Building per-robot ROS–Gazebo bridges

Independent control depends on namespaced topics. Without that, all robots would try to share the same command and feedback channels. The function below generates one bridge configuration per robot:

```python
def _make_bridge_yaml(robot_name, world_name, model_name):
    bridge_config = [
        {
            'ros_topic_name': f'/{robot_name}/joint_trajectory',
            'gz_topic_name': f'/model/{robot_name}/joint_trajectory',
            'ros_type_name': 'trajectory_msgs/msg/JointTrajectory',
            'gz_type_name': 'gz.msgs.JointTrajectory',
            'direction': 'ROS_TO_GZ',
        },
        {
            'ros_topic_name': f'/{robot_name}/joint_states',
            'gz_topic_name': f'/world/{world_name}/model/{robot_name}/joint_state',
            'ros_type_name': 'sensor_msgs/msg/JointState',
            'gz_type_name': 'gz.msgs.Model',
            'direction': 'GZ_TO_ROS',
        },
        {
            'ros_topic_name': f'/{robot_name}/joint_trajectory_progress',
            'gz_topic_name': f'/model/{robot_name}/joint_trajectory_progress',
            'ros_type_name': 'std_msgs/msg/Float64',
            'gz_type_name': 'gz.msgs.Double',
            'direction': 'GZ_TO_ROS',
        },
    ]

    if model_name in MOBILE_MODELS:
        bridge_config.extend([
            {
                'ros_topic_name': f'/{robot_name}/cmd_vel',
                'gz_topic_name': f'/model/{robot_name}/cmd_vel',
                'ros_type_name': 'geometry_msgs/msg/Twist',
                'gz_type_name': 'gz.msgs.Twist',
                'direction': 'ROS_TO_GZ',
            },
            {
                'ros_topic_name': f'/{robot_name}/odom',
                'gz_topic_name': f'/model/{robot_name}/odom',
                'ros_type_name': 'nav_msgs/msg/Odometry',
                'gz_type_name': 'gz.msgs.Odometry',
                'direction': 'GZ_TO_ROS',
            },
            {
                'ros_topic_name': f'/{robot_name}/tf',
                'gz_topic_name': f'/model/{robot_name}/tf',
                'ros_type_name': 'tf2_msgs/msg/TFMessage',
                'gz_type_name': 'gz.msgs.Pose_V',
                'direction': 'GZ_TO_ROS',
            },
        ])
```

Listing 3.3: Generation of robot-specific ROS–Gazebo bridge topics.

This excerpt is important for three reasons:

1. it maps each robot's `/joint_trajectory` command into its own Gazebo transport topic;
2. it maps each robot's `/joint_states` feedback into its own ROS topic;
3. it conditionally adds mobile-base topics when the model belongs to the `MOBILE_MODELS` set.

That logic is the main reason why the final workspace can support several robots at once without topic collisions.

3.4 Step 4: Spawning robots automatically from the registry

Once the registry and bridging mechanisms existed, the launch file could iterate over the list of enabled robots and create the required runtime actions automatically. The following excerpt is the central deployment loop of the final project:

```
    for robot in robots:
        robot_name = str(robot['name'])
        model_name = str(robot.get('model', 'kuka_kr6r900sixx'))
        x_pose = float(robot.get('x_pose', 0.0))
        y_pose = float(robot.get('y_pose', 0.0))
        z_pose = float(robot.get('z_pose', 0.0))
        yaw = float(robot.get('yaw', 0.0))
        model_sdf, urdf_path = _resolve_robot_assets(description_pkg_path, model_name)
        spawn_sdf = model_sdf
        frame_prefix = '' if model_name in MOBILE_MODELS else f'{robot_name}/'

        if model_name in MOBILE_MODELS:
            spawn_sdf = _materialize_mobile_model_sdf(model_sdf, robot_name)

        if urdf_path not in robot_descriptions:
            with open(urdf_path, 'r', encoding='utf-8') as infp:
                robot_descriptions[urdf_path] = infp.read()

        bridge_file = _make_bridge_yaml(robot_name, world_entity_name, model_name)

        spawn_actions.extend([
            Node(
                package='ros_gz_sim',
                executable='create',
                name=f'spawn_{robot_name}',
                output='screen',
                parameters=[{
                    'world': world_entity_name,
                    'file': spawn_sdf,
                    'name': robot_name,
                    'allow_renaming': False,
                    'x': x_pose,
                    'y': y_pose,
                    'z': z_pose,
                    'Y': yaw,
                }],
            ),
            Node(
                package='robot_state_publisher',
                executable='robot_state_publisher',
                namespace=robot_name,
                output='screen',
                remappings=[('/tf', 'tf'), ('/tf_static', 'tf_static')],
                parameters=[{
                    'use_sim_time': use_sim_time,
                    'robot_description': robot_descriptions[urdf_path],
                    'frame_prefix': frame_prefix,
                }],
            ),
            Node(
                package='ros_gz_bridge',
                executable='parameter_bridge',
                name=f'{robot_name}_bridge',
                output='screen',
                arguments=['--ros-args', '-p', f'config_file:={bridge_file}'],
            ),
        ])
```

Listing 3.4: Per-robot spawn logic, state publisher creation, and bridge launch.

This loop is where the project stopped being "a KUKA integration" and became "a multi-robot framework." Each enabled entry in the YAML file now produces:

- one entity spawn action;
- one `robot_state_publisher`;
- one bridge instance;
- one namespaced ROS interface.

3.5 Step 5: Adding the Stäubli TX2-60L

After the launch was generalized, the Stäubli TX2-60L became the first proof that the architecture was reusable. The robot was integrated as its own articulated SDF model and URDF pair, then activated through the YAML registry.

The benefit of this step is architectural rather than merely visual. The robot was added without restructuring the launch file, which confirmed that the generic spawn system was valid. In practice, the Stäubli model follows the same final pattern established by KUKA: articulated links, explicit joints, collision meshes, inertials, a joint-state publisher, and a joint-trajectory controller.

3.6 Step 6: Adding Yaskawa HC10 and HC10DT as distinct robots

The Yaskawa phase tested whether two similar robots from the same family could coexist without being confused. The HC10 and HC10DT have similar structure, but the final solution needed to keep their resources genuinely separate.

The difference appears clearly near the wrist-end resources of the DT model. The HC10 uses the standard HC10 wrist resources:

- `model://yaskawa_hc10/meshes/hc10/visual/link_6_t.dae`
- `model://yaskawa_hc10/meshes/hc10/collision/link_6_t.stl`

The HC10DT uses its own DT-specific wrist resources:

- `model://yaskawa_hc10dt/meshes/hc10dt/visual/link_6_t.dae`
- `model://yaskawa_hc10dt/meshes/hc10dt/collision/link_6_t.stl`

This distinction is important because it proves that the final configuration does not simply duplicate one Yaskawa robot under two names. The DT variant remains a genuinely separate model in the final workspace.

3.7 Step 7: Adding TIAGo first as a manipulator

TIAGo required a different approach because it is not a fixed-base industrial arm. It combines a mobile platform, a prismatic torso, a seven-joint arm, and a head. The first integration target was the manipulator part: torso, arm, and head had to respond to the same `/joint_trajectory` interface used by the industrial robots.

The torso and arm part of the TIAGo model is built around a prismatic `torso_lift_joint` followed by the articulated arm chain beginning with `arm_1_joint`. In the final file, these joints are defined in `mfja_3rd_floor_description/models/tiago/model.sdf` with explicit poses, limits, and damping values. This step made TIAGo compatible with the rest of the control framework. Even before base mobility was added, the robot could already publish joint states and accept trajectory commands in its namespace.

3.8 Step 8: Upgrading TIAGo to a real mobile manipulator

The next critical improvement was to make TIAGo move through the map, not only move its arm. This required two coordinated changes:

- the SDF model had to include a differential-drive plugin;
- the launch file had to bridge the mobile topics in addition to the arm topics.

The mobile plugin block is shown below:

```
    <plugin
      filename="gz-sim-diff-drive-system"
      name="gz::sim::systems::DiffDrive">
      <left_joint>left_wheel_joint</left_joint>
      <right_joint>right_wheel_joint</right_joint>
      <wheel_separation>0.46</wheel_separation>
      <wheel_radius>0.10</wheel_radius>
      <wheel_torque>120</wheel_torque>
      <max_linear_acceleration>1.0</max_linear_acceleration>
      <topic>cmd_vel</topic>
      <odom_topic>odom</odom_topic>
      <tf_topic>tf</tf_topic>
      <frame_id>odom</frame_id>
      <child_frame_id>base_footprint</child_frame_id>
      <odom_publisher_frequency>50</odom_publisher_frequency>
    </plugin>

    <plugin
      filename="gz-sim-joint-state-publisher-system"
      name="gz::sim::systems::JointStatePublisher">
      <joint_name>left_wheel_joint</joint_name>
      <joint_name>right_wheel_joint</joint_name>
      <joint_name>torso_lift_joint</joint_name>
      <joint_name>arm_1_joint</joint_name>
      <joint_name>arm_2_joint</joint_name>
      <joint_name>arm_3_joint</joint_name>
      <joint_name>arm_4_joint</joint_name>
      <joint_name>arm_5_joint</joint_name>
      <joint_name>arm_6_joint</joint_name>
      <joint_name>arm_7_joint</joint_name>
      <joint_name>head_1_joint</joint_name>
      <joint_name>head_2_joint</joint_name>
    </plugin>

    <plugin
      filename="gz-sim-joint-trajectory-controller-system"
      name="gz::sim::systems::JointTrajectoryController">
```

Listing 3.5: TIAGo mobility, joint-state publication, and joint-trajectory control plugins.

The launch file complements this with one more important helper:

```python
def _materialize_mobile_model_sdf(model_sdf_path, robot_name):
    with open(model_sdf_path, 'r', encoding='utf-8') as infp:
        sdf_text = infp.read()

    replacements = {
        '<topic>cmd_vel</topic>': f'<topic>/model/{robot_name}/cmd_vel</topic>',
        '<odom_topic>odom</odom_topic>': f'<odom_topic>/model/{robot_name}/odom</odom_topic>',
        '<tf_topic>tf</tf_topic>': f'<tf_topic>/model/{robot_name}/tf</tf_topic>',
    }

    for source, target in replacements.items():
        if source not in sdf_text:
            raise RuntimeError(
                f'Expected token "{source}" not found in mobile model SDF: {model_sdf_path}'
            )
        sdf_text = sdf_text.replace(source, target, 1)

    output_path = os.path.join(
        tempfile.gettempdir(), f'{robot_name}_mobile_model.sdf')
    with open(output_path, 'w', encoding='utf-8') as outfp:
        outfp.write(sdf_text)

    return output_path
```

Listing 3.6: Runtime rewriting of TIAGo mobile topic names per robot instance.

This pair of changes is what made `tiago1` a true mobile manipulator. The arm still uses `/tiago1/joint_trajectory`, while the base is driven through `/tiago1/cmd_vel` and publishes `/tiago1/odom` and `/tiago1/tf`.

3.9 Step 9: Enriching the world with industrial station assets

After the robot integrations were stable, the map was extended with the station tables and industrial support parts that correspond to the robot positions. These additions belong in the world file because they are static environment elements rather than articulated robots.

The corresponding static placements were added directly to `mfja_3rd_floor_description/worlds/` The final world now includes the KUKA table, the Stäubli table, two Yaskawa tables, the left and right carters, both Montratec rails, and four shuttle elements. This work improved the final simulation in three ways:

- the robot stations became physically meaningful rather than floating over an empty floor;
- robot `z_pose` values could be aligned with actual support tables;
- the world became closer to a realistic lab cell instead of a simple model demo.

3.10 Step 10: Adding dedicated room assets and a room-315-only mode

Once the complete third-floor simulation was stable, a second execution mode became necessary: a lighter environment dedicated to robot and control experiments in room 315. This requirement could not be satisfied cleanly by overloading the full-floor world with many runtime conditionals. The final solution was therefore to add:

- a dedicated room mesh based on `315_room.stl`;
- a separate world file `mfja_3rd_floor_description/worlds/room_315_only.world`;
- a separate bringup package `mfja_room_315_bringup`;
- a separate robot registry `mfja_robot_control_config/config/robots_room_315_only.yaml`.

This new mode reuses the same shared robot models, URDF files, and generic spawn logic, but it replaces the full-floor world with a dedicated room 315 scene. The accepted final room-315-only scene keeps the room mesh itself, the four robot support tables, and the main industrial fixtures such as the carters, rails, and shuttles, while leaving out general lab furniture and room-314 assets. This makes the room-only mode lighter and cleaner while preserving the robot infrastructure needed for experiments.

3.11 Step 11: Refactoring the repository into five ROS 2 packages

The final structural step was the meta-repository refactor. Earlier stages still lived in one large package. That was workable for one developer, but it would create unnecessary Git conflicts once environment assets, launch logic, room 315 bringup, and robot-control configuration started evolving in parallel.

The repository was therefore split into five packages with clear responsibilities:

- `mfja_3rd_floor_gz`: umbrella and compatibility package that preserves simple launch commands under the historical project name;
- `mfja_3rd_floor_description`: shared meshes, models, URDF files, and world files;
- `mfja_robot_control_config`: shared launch logic, robot YAML files, and synchronized command tools;
- `mfja_3rd_floor_bringup`: the full-floor launch entry points;
- `mfja_room_315_bringup`: the room-315-only launch entry point.

This change preserved the working behavior but improved long-term maintainability. Environment work can now evolve in the description package, runtime control logic can evolve in the configuration package, and the two bringup packages can change independently according to whether a developer is working on the complete floor or only on room 315.

3.12 Step 12: Freezing the final scope

Experimental SCARA work was explored during development, but it was removed from the accepted final configuration. That cleanup matters in the report because it clarifies the scope actually delivered. The final system is therefore limited intentionally to:

- KUKA KR6;

- Stäubli TX2-60L;
- Yaskawa HC10;
- Yaskawa HC10DT;
- TIAGo;
- the station furniture and industrial static assets that remain in the world.

Chapter 4

Important Final Code and Why It Matters

4.1 Code block 1: The configuration file

Listing 2.1 is small, but it is central to the project. The same field structure is also reused in `mfja_robot_control_config/config/robots_room_315_only.yaml`. Each field directly affects runtime behavior:

- `name` becomes both the ROS namespace and the Gazebo entity name;
- `model` selects the SDF folder and matching URDF;
- `x_pose`, `y_pose`, `z_pose`, and `yaw` define reproducible placement;
- `enabled` decides whether the robot participates in the launch.

This file is the reason the final setup is manageable. It allows placements and robot activation to change without editing the launch implementation.

4.2 Code block 2: Bridge generation

Listing 3.3 is the main control-isolation mechanism. It converts a generic concept, "this robot has trajectory and joint-state topics," into robot-specific names:

- `/kuka1/joint_trajectory`
- `/staubli1/joint_trajectory`
- `/yaskawa_hc10_1/joint_trajectory`
- `/yaskawa_hc10dt_1/joint_trajectory`
- `/tiago1/joint_trajectory`

Without this logic, multiple robots could not be controlled independently from the same ROS 2 environment.

4.3 Code block 3: Spawn loop and state publishers

Listing 3.4 contains the actual deployment algorithm. The launch file:

1. loads the world;
2. sets Gazebo model and resource paths;
3. iterates over enabled robots;
4. resolves model assets;
5. creates a bridge file;
6. spawns the model;
7. starts one `robot_state_publisher`;
8. starts one bridge instance.

This is the core of the final architecture because it removed the need for robot-specific launch duplication.

4.4 Code block 4: KUKA articulation

The KUKA excerpts in Listings 3.1 and 3.2 explain why the final KUKA model works correctly. The final implementation uses:

- child links placed relative to joints;
- explicit parent-child joint relations;
- axis definitions consistent with the robot chain;
- realistic limits, damping, and friction.

The same model file also contains the joint-state publisher and joint-trajectory controller plugins. These complete the robot by turning the articulated geometry into a controllable Gazebo entity.

4.5 Code block 5: TIAGo mobility support

TIAGo needed a separate treatment because it mixes two control modes:

- trajectory control for torso, arm, and head;
- velocity control for the mobile base.

The plugin excerpt in Listing 3.5 plus the topic-rewriting function in Listing 3.6 are the two code blocks that enable this mixed behavior. Together they explain why the TIAGo arm can move independently while the base also responds to `cmd_vel`.

4.6 Code block 6: Static world composition

The world file belongs to the static-scene layer of the architecture. It contains the objects that shape the robot workstations:

- KUKA table;
- Stäubli table;
- two Yaskawa tables;
- left and right carters;
- right and left Montratec rails;
- four shuttle elements.

Those placements matter because the final robot poses in the robot YAML files were chosen relative to these station assets. In the full-floor mode, this relationship is defined against `mfja_3rd_floor_description/worlds/mfja_3rd_floor.world`. In the room-315-only mode, the dedicated world keeps only the room 315 mesh, the robot support tables, and the industrial fixtures needed for experiments.

4.7 Code block 7: Launch-time robot selection and synchronized demos

The final workspace also gained two practical operational improvements after the original five-robot integration was stable.

First, the launch file now accepts a `robots` argument. This allows an operator to type, for example, `robots:=kuka,tiago` or `robots:=1,5` and spawn only the requested instances, even if more robots are enabled in the YAML file. This is useful during testing because it avoids repeated editing of `mfja_robot_control_config/config/robots.yaml` for short experiments and makes quick selection much easier from the command line.

Second, the script `mfja_robot_control_config/scripts/multi_robot_sync_demo.py` provides one coordinated ROS 2 process that can publish trajectories to several robots from one node. This is a better workflow than launching many independent `ros2 topic pub` commands from separate shells, because all commands are emitted from one process after subscriber readiness has been verified. In normal operation, the operator first launches the desired simulation mode in Terminal 1 and then uses the synchronized tool in Terminal 2 only for the robots that should actually move.

Chapter 5

How Independent Control Works

5.1 Namespacing strategy

Independent control is based on a simple rule: every robot is identified by a unique name, and that same name is used as its ROS namespace and Gazebo model name. For example:

- `kuka1`
- `staubli1`
- `yaskawa_hc10_1`
- `yaskawa_hc10dt_1`
- `tiago1`

This leads directly to separate control topics. A trajectory sent to `/kuka1/joint_trajectory` does not affect any other robot because the bridge maps it only to `/model/kuka1/joint_trajectory`. The same is true for Stäubli, both Yaskawa robots, and TIAGo.

5.2 Why one state publisher per robot was necessary

TF publication also had to stay isolated. The final launch therefore starts one `robot_state_publisher` instance per robot. This prevents the frame trees from collapsing into one ambiguous set of joints and links. In the fixed-base industrial arms, a `frame_prefix` is used so that the TF tree remains clearly tied to the robot instance. For TIAGo, the mobile model path is treated specially because its base-related topics and frames are already managed in a different way.

5.3 Why TIAGo needed extra logic

The industrial arms share a common control style: joint trajectories only. TIAGo does not. It contains:

- wheel joints and a differential-drive controller;
- a prismatic torso;

- a seven-joint arm;
- a pan/tilt head.

The final architecture remained generic, but not blindly generic. The `MOBILE_MODELS` set marks which model requires additional bridging and topic rewriting. This is a practical engineering choice: the system remains extensible while still recognizing that not all robots are controlled in the same way.

Chapter 6

Validation and User Operation Guide

6.1 What was validated in the final system

The final result was validated operationally through repeated checks:

- each robot could spawn using the final YAML registry;
- each industrial arm reacted to a namespaced `/joint_trajectory` command;
- TIAGo reacted both to `/tiago1/joint_trajectory` and `/tiago1/cmd_vel`;
- the world loaded the static station assets without breaking robot placement;
- duplicate robot control did not occur because topic names were isolated.

6.2 How to run the program

The recommended workflow uses two terminals. Terminal 1 starts the simulation. Terminal 2 is used for checking topics and sending commands. The same ROS environment must be sourced in every terminal before running any command.

Terminal 1: build and launch the full floor

Use the following commands exactly:

```
export MFJA_WS=~/mfja_3rd_floor_ws
cd "$MFJA_WS"
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
source install/setup.bash
ros2 launch mfja_3rd_floor_bringup full_floor.launch.py
```

Listing 6.1: Final launch procedure.

When this command finishes starting, Gazebo should contain the enabled robots from `mfja_robot_control_config/config/robots.yaml`. In the current final configuration these are `kuka1`, `staubli1`, `yaskawa_hc10_1`, `yaskawa_hc10dt_1`, and `tiago1`. For compatibility, the same mode also remains available through `ros2 launch mfja_3rd_floor_gz full_floor.launch.py`.

Alternative launch: room 315 only

The room-315-only mode is launched from a different bringup package and uses a separate robot registry:

```
export MFJA_WS=~/mfja_3rd_floor_ws
cd "$MFJA_WS"
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch mfja_room_315_bringup room_315_only.launch.py
```

Listing 6.2: Launching the room-315-only mode.

This mode loads `mfja_3rd_floor_description/worlds/room_315_only.world` and the dedicated robot registry `mfja_robot_control_config/config/robots_room_315_only.yaml`. It is intended for lighter, cleaner robot experiments than the full third-floor world. In its final accepted form it keeps only room 315, the four robot support tables, and the main industrial fixtures such as the carters, rails, and shuttles. For compatibility, the same launch remains available through `ros2 launch mfja_3rd_floor_gz room_315_only.launch.py`.

Optional launch-time robot selection

When only a subset of robots is needed for a quick experiment, the launch file can now override the YAML activation set directly from the command line:

```
ros2 launch mfja_3rd_floor_bringup full_floor.launch.py \
  robots:=kuka,tiago

ros2 launch mfja_3rd_floor_bringup full_floor.launch.py \
  robots:=1,5
```

Listing 6.3: Spawning only selected robots from the command line.

This argument now accepts three convenient forms:

- full robot names such as `kuka1,tiago1`;
- short aliases such as `kuka,staubli,hc10,hc10dt,tiago`;
- numeric indices by YAML order such as `1,5`.

In the current registry order, the shortcuts are:

- `1` or `kuka` → `kuka1`;
- `2` or `staubli` → `staubli1`;
- `3` or `hc10` → `yaskawa_hc10_1`;
- `4` or `hc10dt` → `yaskawa_hc10dt_1`;
- `5` or `tiago` → `tiago1`.

If the argument is omitted, the launch file falls back to the `enabled` flags in YAML. If the operator uses `robots:=all`, every robot listed in the YAML registry is spawned, even if some entries are currently disabled there.

Terminal 2: source the workspace for control

Open a second terminal and source the environment again:

```
export MFJA_WS=~/mfja_3rd_floor_ws
cd "$MFJA_WS"
source /opt/ros/jazzy/setup.bash
source install/setup.bash
```

Listing 6.4: Second terminal preparation.

6.3 How to verify that the system is ready

Before sending motion commands, it is good practice to verify that the robots and their topics are available. The following checks are sufficient:

```
ros2 topic list | rg '^/(kuka1|staubli1|yaskawa_hc10_1|yaskawa_hc10dt_1|tiago1)/'

ros2 topic info /kuka1/joint_trajectory
ros2 topic info /staubli1/joint_trajectory
ros2 topic info /yaskawa_hc10_1/joint_trajectory
ros2 topic info /yaskawa_hc10dt_1/joint_trajectory
ros2 topic info /tiago1/joint_trajectory
ros2 topic info /tiago1/cmd_vel
```

Listing 6.5: Basic runtime checks before sending commands.

If a topic shows at least one subscription on the command side, the bridge and controller path are active. If a command waits forever for a subscriber, the robot was either not spawned, not enabled in the YAML file, or the launch is not fully ready yet.

6.4 How to inspect the synchronized multi-robot tool

The final repository now includes `mfja_robot_control_config/scripts/multi_robot_sync_demo.py` which can publish coordinated commands to all enabled robots from one ROS 2 process. Before using custom goals, it is useful to print the expected joint names and default target vectors:

```
export MFJA_WS=~/mfja_3rd_floor_ws
cd "$MFJA_WS"
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 run mfja_robot_control_config multi_robot_sync_demo.py --list-joints
```

Listing 6.6: Listing the joint order expected by the synchronized demo tool.

This command prints each enabled robot name, its model type, the exact joint order expected by the tool, and the current default position vector built into the script. In normal use there is no need to pass an additional robot-registry argument here: the standard workflow is simply to launch the desired simulation mode first and then inspect or command the robots from Terminal 2.

6.5 How to choose between one-by-one control and synchronized control

There are now two different command styles in the final repository, and they should not be mixed conceptually:

- **One-by-one control**: send one command directly to one robot topic. This is the best method for debugging, for checking one controller, or for moving only one robot while leaving all others untouched.
- **Synchronized control**: use `multi_robot_sync_demo.py`. This is the best method when two, three, or more robots must begin one operation together from one ROS 2 node.

The practical difference is important. A direct `ros2 topic pub -once` command addresses only the topic written in that command. By contrast, the synchronized tool can either run its built-in full demo posture set or publish only to the robots explicitly named by `-goal`. Therefore, if only two or three robots should move together while the others must remain completely inactive, the recommended workflow is:

1. if no `-goal` arguments are provided, the tool commands all enabled robots that have built-in presets;
2. if one or more `-goal` arguments are provided, only the robots explicitly named by those arguments are commanded;
3. the `robots:=...` launch argument is still useful when the operator wants to reduce scene clutter or spawn only a temporary subset in Gazebo.

This distinction prevents a common misunderstanding: the synchronized tool now uses two modes. With no `-goal` arguments, it runs the full built-in synchronized demo for all enabled robots. With one or more `-goal` arguments, it publishes only to the robots listed on the command line.

6.6 How to control the industrial robots

The fixed-base industrial robots all use the same control method: publish one `trajectory_msgs/msg/Jo` message to the robot-specific `/joint_trajectory` topic. The message must use the correct joint names and straight ASCII quotes. Typographic quotes copied from formatted text must not be used.

KUKA KR6

KUKA uses the joints `joint_a1` to `joint_a6`.

```
ros2 topic pub --once /kuka1/joint_trajectory \
trajectory_msgs/msg/JointTrajectory \
"{joint_names: ['joint_a1','joint_a2','joint_a3','joint_a4','joint_a5','joint_a6'], \
points: [{positions: [0.0,-0.8,1.2,0.0,0.6,0.0], \
time_from_start: {sec: 3, nanosec: 0}}]}"
```

Listing 6.7: Example command for KUKA.

Stäubli TX2-60L

Stäubli uses the joints `joint_1` to `joint_6`.

```
ros2 topic pub --once /staubli1/joint_trajectory \
trajectory_msgs/msg/JointTrajectory \
"{joint_names: ['joint_1','joint_2','joint_3','joint_4','joint_5','joint_6'], \
points: [{positions: [0.0,0.3,-0.5,0.0,0.6,0.0], \
time_from_start: {sec: 3, nanosec: 0}}]}"
```

Listing 6.8: Example command for Stäubli.

Yaskawa HC10

The Yaskawa HC10 uses the joints `joint_1_s`, `joint_2_l`, `joint_3_u`, `joint_4_r`, `joint_5_b`, and `joint_6_t`.

```
ros2 topic pub --once /yaskawa_hc10_1/joint_trajectory \
trajectory_msgs/msg/JointTrajectory \
"{joint_names: ['joint_1_s','joint_2_l','joint_3_u','joint_4_r','joint_5_b','joint_6_t'], \
points: [{positions: [0.0,-0.6,0.8,0.0,0.5,0.0], \
time_from_start: {sec: 3, nanosec: 0}}]}"
```

Listing 6.9: Example command for Yaskawa HC10.

Yaskawa HC10DT

The HC10DT uses the same joint names as the HC10, but it remains a separate robot because it has its own model and namespace.

```
ros2 topic pub --once /yaskawa_hc10dt_1/joint_trajectory \
trajectory_msgs/msg/JointTrajectory \
"{joint_names: ['joint_1_s','joint_2_l','joint_3_u','joint_4_r','joint_5_b','joint_6_t'], \
points: [{positions: [0.0,-0.5,0.7,0.0,0.4,0.0], \
time_from_start: {sec: 3, nanosec: 0}}]}"
```

Listing 6.10: Example command for Yaskawa HC10DT.

6.7 How to control TIAGo

TIAGo must be treated as two subsystems:

- the torso, arm, and head use `/tiago1/joint_trajectory`;
- the mobile base uses `/tiago1/cmd_vel`.

TIAGo arm, torso, and head

The joint names are `torso_lift_joint`, `arm_1_joint` to `arm_7_joint`, and `head_1_joint`, `head_2_joint`.

```
ros2 topic pub --once /tiago1/joint_trajectory \
trajectory_msgs/msg/JointTrajectory \
"{joint_names: ['torso_lift_joint','arm_1_joint','arm_2_joint','arm_3_joint', \
'arm_4_joint','arm_5_joint','arm_6_joint','arm_7_joint','head_1_joint','head_2_joint'], \
points: [{positions: [0.10,0.3,-0.5,-0.4,1.0,0.2,-0.2,0.1,0.2,-0.2], \
time_from_start: {sec: 4, nanosec: 0}}]}"
```

Listing 6.11: Example command for the TIAGo manipulator chain.

TIAGo mobile base

The mobile base must receive velocity commands continuously while it is moving. A single `-once` command is usually too short to produce visible motion. For that reason, the correct operational command uses `-r 20` or another non-zero publish rate:

```
ros2 topic pub -r 20 /tiago1/cmd_vel geometry_msgs/msg/Twist \
"{linear: {x: 0.25, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.35}}"
```

Listing 6.12: Example command for TIAGo base motion.

Stopping TIAGo

To stop TIAGo safely, either interrupt the continuous publisher with `Ctrl+C` or publish a zero-velocity command:

```
ros2 topic pub --once /tiago1/cmd_vel geometry_msgs/msg/Twist \
"{linear: {x: 0.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
```

Listing 6.13: Stopping the TIAGo base.

Monitoring TIAGo odometry

The base state can be monitored using:

```
ros2 topic echo /tiago1/odom
```

Listing 6.14: Reading TIAGo odometry.

6.8 How to move one robot at a time

If the objective is to move one robot while all other robots remain untouched, the correct method is to publish directly to that robot namespace. The procedure is simple:

1. identify the robot name;
2. identify the correct joint order;
3. publish one `JointTrajectory` message to that robot only.

For example, to move only KUKA, use only `/kuka1/joint_trajectory`. To move only Stäubli, use only `/staubli1/joint_trajectory`. The commands shown earlier in this chapter are therefore not just examples of syntax; they are also the correct operational method for isolated robot control.

The same logic applies to TIAGo:

- if only the manipulator should move, publish only to `/tiago1/joint_trajectory`;
- if only the mobile base should move, publish only to `/tiago1/cmd_vel`.

6.9 How to command several robots together from one node

Independent single-robot commands remain useful for debugging, but they are not the best solution when several robots must move together in one operation. If five separate shell commands are launched with `&`, they start only approximately at the same time. The final package therefore includes a synchronized coordinator script.

Recommended procedure for two or three robots moving together

If the goal is to move only two robots together, or only three robots together, the synchronized tool can now target that subset directly. In other words, if the operator provides `-goal` only for `kuka1` and `staubli1`, then only those two robots move, even if more robots are currently spawned in Gazebo.

```
ros2 run mfja_robot_control_config multi_robot_sync_demo.py \
  --goal kuka1=1.2,-1.2,1.4,0.0,0.3,0.0 \
  --goal staubli1=0.1,0.4,-0.6,0.0,0.5,0.0 \
  --trajectory-duration 4.0
```

Listing 6.15: Synchronized motion for two robots only.

The exact same principle extends to three robots:

```
ros2 run mfja_robot_control_config multi_robot_sync_demo.py \
  --goal kuka1=1.2,-1.2,1.4,0.0,0.3,0.0 \
  --goal staubli1=0.1,0.4,-0.6,0.0,0.5,0.0 \
  --goal yaskawa_hc10_1=0.0,-0.7,0.9,0.0,0.4,0.0 \
  --trajectory-duration 4.0
```

Listing 6.16: Synchronized motion for three robots only.

Using `robots:=...` during launch is still a good workflow when the operator wants a smaller scene for testing, but it is no longer mandatory just to restrict synchronized motion to two or three robots.

Approximate simultaneous control from the shell

It is still possible to send several `ros2 topic pub -once` commands from the shell using multiple terminals or shell background operators. However, this only gives approximate simultaneity because each shell process starts at a slightly different time. That method is acceptable for quick visual tests, but it is not the recommended final workflow for coordinated multi-robot operation.

Synchronized example with default built-in goals

The following command publishes one coordinated trajectory set to every enabled robot in the YAML file:

```
export MFJA_WS=~/mfja_3rd_floor_ws
cd "$MFJA_WS"
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 run mfja_robot_control_config multi_robot_sync_demo.py
```

Listing 6.17: Coordinated multi-robot motion using the built-in default goals.

Synchronized example with custom goals from the command line

The same tool can now override the target vector of each robot directly from the command line using repeated `-goal` arguments:

```
ros2 run mfja_robot_control_config multi_robot_sync_demo.py \
  --goal kuka1=1.2,-1.2,1.4,0.0,0.3,0.0 \
  --goal staubli1=0.1,0.4,-0.6,0.0,0.5,0.0 \
  --goal yaskawa_hc10_1=0.0,-0.7,0.9,0.0,0.4,0.0 \
  --goal yaskawa_hc10dt_1=0.0,-0.4,0.6,0.0,0.3,0.0 \
  --goal tiago1=0.10,0.2,-0.4,-0.3,0.9,0.1,-0.1,0.0,0.1,-0.1 \
  --trajectory-duration 4.0
```

Listing 6.18: Coordinated motion with custom target vectors for all robots.

The values after each `-goal` argument must match the joint order printed by Listing 6.6. If the number of values does not match the number of joints expected for that robot, the tool exits with a clear error message instead of publishing a malformed command.

Returning all robots to the script defaults

The same utility can also be used to return all robots to the current default demonstration posture by explicitly restating the built-in vectors:

```
ros2 run mfja_robot_control_config multi_robot_sync_demo.py \
  --goal kuka1=0.8,-1.0,1.2,0.0,0.4,0.0 \
  --goal staubli1=0.0,0.3,-0.5,0.0,0.6,0.0 \
  --goal yaskawa_hc10_1=0.0,-0.6,0.8,0.0,0.5,0.0 \
  --goal yaskawa_hc10dt_1=0.0,-0.5,0.7,0.0,0.4,0.0 \
  --goal tiago1=0.10,0.3,-0.5,-0.4,1.0,0.2,-0.2,0.1,0.2,-0.2 \
  --trajectory-duration 4.0
```

Listing 6.19: Returning the full cell to the synchronized default posture.

Adding TIAGo base motion to the synchronized scenario

The synchronized tool can also command TIAGo base motion while the manipulator trajectories are being published:

```
ros2 run mfja_robot_control_config multi_robot_sync_demo.py \
  --goal kuka1=1.2,-1.2,1.4,0.0,0.3,0.0 \
  --goal staubli1=0.1,0.4,-0.6,0.0,0.5,0.0 \
  --goal yaskawa_hc10_1=0.0,-0.7,0.9,0.0,0.4,0.0 \
  --goal yaskawa_hc10dt_1=0.0,-0.4,0.6,0.0,0.3,0.0 \
  --goal tiago1=0.10,0.2,-0.4,-0.3,0.9,0.1,-0.1,0.0,0.1,-0.1 \
  --trajectory-duration 4.0 \
  --tiago-base-linear 0.20 \
  --tiago-base-angular 0.25 \
  --tiago-base-duration 4.0
```

Listing 6.20: Coordinated arm trajectories with TIAGo base motion.

This does not create hard real-time synchronization in the strict control-theory sense, but it is operationally superior to launching many independent CLI publishers because all commands are emitted from one ROS node after subscriber readiness has been checked.

6.10 How to change robot positions and active robots

The placement and activation of robots is controlled from the robot YAML files:

- `mfja_robot_control_config/config/robots.yaml` for the complete third-floor simulation;
- `mfja_robot_control_config/config/robots_room_315_only.yaml` for the dedicated room-315-only mode.

Each entry contains:

- `name`: robot namespace and Gazebo entity name;
- `model`: which SDF and URDF pair to load;
- `x_pose`, `y_pose`, `z_pose`: Cartesian placement in the world;
- `yaw`: heading angle around the vertical axis;
- `enabled`: whether that robot is spawned.

A typical modification workflow is:

1. choose the correct YAML file for the mode you are using;
2. edit `mfja_robot_control_config/config/robots.yaml` or `mfja_robot_control_config/conf`
3. save the new `x_pose`, `y_pose`, `z_pose`, `yaw`, or `enabled` values;
4. stop the current launch;
5. launch the simulation again.

If only the YAML file changed, no package rebuild is normally required because the launch command already points to the configuration file directly. Rebuild is required only when Python launch code, SDF files, or URDF files are modified.

If the operator does not want to edit YAML at all for a quick test, the new `robots` launch argument can temporarily override the selection from the command line, for example `robots:=kuka,tiago` or `robots:=1,5`.

6.11 Typical issues and final operational notes

Several issues appeared during development. The final code contains the fixes that solved them, and these notes help during operation:

- **Missing or visually separated links**: fixed in the final models by using explicit child-link placement relative to joints and by correcting resource paths.
- **Robot duplication or wrong spawning behavior**: avoided by unique robot names and the launch partition handling.
- **Robots at wrong height**: corrected through `z_pose` in the robot YAML files and by adding the corresponding station tables in the world.
- **Need to test only one or two robots quickly**: use the launch argument `robots:=kuka,tiago` or `robots:=1,5` instead of rewriting the YAML activation flags.
- **TIAGo arm moves but base does not**: the base requires the `cmd_vel` publisher to run continuously for a short time; a one-shot message is often not enough.
- **Need coordinated motion across several robots**: use `multi_robot_sync_demo.py` rather than launching many separate `ros2 topic pub` commands from different shells.
- **Command copied with wrong quotes**: always use plain ASCII quotes exactly as written in the code blocks of this report.
- **New packages not found after the refactor**: remove stale build and install products from the old single-package layout, rebuild the four new packages, and source `install/setup.bash` again before launching.

Chapter 7

Conclusion

The final project evolved from a narrow task, adding one KUKA robot, into a reusable multi-robot simulation framework for Gazebo Harmonic. The most important result is not any single model file by itself. The most important result is the architecture that now connects configuration, spawning, bridging, TF publication, and world composition into one coherent system.

The KUKA integration established the articulation and controller pattern. The YAML-driven launch architecture made future additions scalable. The Stäubli and Yaskawa integrations demonstrated that the framework was reusable across industrial robot families. TIAGo then proved that the same shared control layer could also support a mobile manipulator through a carefully extended bridge and plugin strategy. The later room-315-only mode demonstrated that the same robot infrastructure could be reused in a lighter experimental scene. Finally, the meta-repository refactor separated shared assets, shared control logic, full-floor bringup, and room-specific bringup into distinct ROS 2 packages while preserving the original functionality.

In summary, the final solution is technically significant for three reasons:

- it is configuration-driven rather than hard-coded;
- it guarantees independent robot control through namespaced interfaces;
- it combines several robot classes, multiple environment modes, and static industrial assets in one reproducible Gazebo Harmonic repository.